

乔卓成

AI Agent / 后端软件工程师

上海 | 华东师范大学 · 信号与信息处理硕士 (2022)

18017822420 | 18017822420@163.com | github.com/tomqiao [zco](https://www.linkedin.com/in/tomqiao/)



个人概述

4 年以上微软软件研发经验，曾任 Software Engineer II，聚焦生产级多智能体系统、Agent Harness 与企业级后端平台。具备 Python、.NET 与 Azure 工程经验，覆盖 Agent Runtime、Context/Memory、工具与沙箱执行、可靠消息、服务化、安全合规及商业化系统；尤其关注概率型 Agent 的可信执行，通过确定性流程控制、独立评价、证据门禁与真实测试执行，缩小模型“声称完成”与任务“可验证完成”之间的差距。

专业技能

Agent 与 AI 系统：Multi-Agent / Sub-Agent、Native Function Calling、Context/Memory、Agentic RAG、Agent Harness、DAG 工作流、独立评价与证据门禁、浏览器与代码执行

后端与分布式系统：Python、FastAPI、C#/.NET、ABP Framework、REST/SSE、Service Bus、Redis、异步任务、幂等与状态机、外部系统集成

云与工程化：Azure、Azure AI Search、Blob、Key Vault、Docker、CI/CD、日志监控与可观测性、自动化测试、可靠性与安全治理

工作经历

微软 | Software Engineer II 2022.06–2026.08

Microsoft Office Agent | 生产级多智能体系统 2025.07–2026.02

面向 M365 用户的生产级通用 Agent 产品，支持复杂任务规划、并发 Sub-Agent、工具与代码执行、本地浏览器操作、长期记忆及 Office 成果物交付；基于 Azure 构建 Agent Runtime、沙箱、数据与消息基础设施，全球发布约两个月达到 **27K WAU**。作为核心开发者，重点负责工具调用协议、长任务可靠性及 Agent 服务化，并参与 Context/Memory、沙箱执行与安全合规体系建设。

- 多智能体 Runtime 与工具调用：**主导将主 Agent/Sub-Agent 的工具协议从文本 XML 迁移至 Native Function Calling，适配 OpenAI/Anthropic 流式协议，统一从工具注册、参数聚合与校验、后端执行到 Tool Result 回填的完整链路；设计 Native/XML 双模式灰度与回退，并修复流式参数残缺、消息序列破坏及进程中断造成的 Tool Call/Result 失配。
- Context、Memory 与 Agentic RAG：**核心参与 Agent 上下文与长期记忆体系建设，通过调用前 Token 统计、分层 Compact 和动态 Skill 换入换出控制长任务上下文，确定性保留任务计划与工具执行状态；参与知识库 Agentic RAG 的架构设计与核心开发，组合关键词、向量与语义排序，形成“候选召回—证据深读—继续检索—原文引用”的 Agentic Loop。
- 沙箱与浏览器执行：**参与构建 Cloud Sandbox + Local Browser 双执行体系，基于隔离沙箱运行模型生成代码，通过预热池、生命周期管理、缓存与分布式锁协调多实例资源；Browser Operator 采用 DOM-first 方式复用用户本地登录态，并在执行后重新观察页面验证结果，支撑 Agent 在真实企业系统中完成任务。
- 长任务可靠性：**独立诊断原有实时消息通道离线不留存、无确认与重投的问题，定义 Agent 长任务的消息留存、Ack、重投及会话保序语义；抽象可灰度、可回退的 MessageTransport，引入支持会话保序与确认重投的可靠消息通道，并保留可回放的过程状态，支撑后端闪断续接与浏览器重连恢复。
- 服务化与安全合规：**独立从 0 交付 Agent Proxy，覆盖代表用户访问下游资源的授权链路、异步任务持久化与重启恢复、容器化及 CI/CD，将 Agent Runtime 封装为可供外部系统集成的生产服务；设计用户级加密与渐进轮换机制，并完成证书安全注入、服务间 HTTPS 与下游证书校验，覆盖存储态和传输态保护。

Microsoft Copilot | 生产级 Agent Harness 与 Loop Engineering 实践 2026.03–2026.08

参与建设并规模化落地面向真实软件交付的多 Agent Harness，建成后成为项目需求实现的主要执行路径：将外部 Issue 转化为持久化 Task，由状态图驱动 Developer、Reviewer、Judge 分阶段完成开发、评审与验收，通过任务领取与超时接管、重试及失败恢复保障长任务可靠运行，并以代码、测试、PR、CI、Review 和截图等证据控制状态流转与合并。

- Graph-Based Harness：**以 Task State 为执行主线，将实现、独立评审、证据裁决、修复迭代与合并建模为可回退的 DAG；Agent 每次只处理当前状态的职责，通过持久化状态与 artifacts 跨 run 交接，避免依赖单次模型上下文完成整条链路。
- Loop Engineering 飞轮：**聚合多次任务的 trace、失败类型、耗时、重试、架构检查与交付证据，识别反复出现的系统性问题；再将结论写回任务模板、Skill、规则、测试或 Harness，并由后续同类任务的执行证据验证改进是否生效，形成“执行—聚合—写回—再验证”的闭环。

工作经历（续）

Office Plus | 商业化平台与身份基础设施

2024.07–2025.06

基于 .NET/ABP 与 Azure 构建的多服务商业化平台，覆盖订单支付、会员与论文权益、增长活动及身份认证，FY25 支撑 **340 万**新增付费订阅、**690 万**美元收入。作为核心后端开发者，参与商业化平台建设，主导第三方能力产品化、周期订阅、渠道直充及身份治理。

- 第三方能力产品化与异步交付**：端到端主导万方论文查重商业化，将第三方 API 扩展为文件上传、订单支付、字数权益、异步检测、结果交付及失败退款的完整闭环；设计 PaperRecord、PaperRight 与消耗 Mapping，采用 SAS 直传 Blob、Service Bus Worker、数据库条件更新和结果 MD5 校验，处理大文件传输、权益超卖、第三方限流及失败补偿。功能上线 **10 天**处理 **6,320 笔**查重订单。
- 订阅、支付与渠道履约**：主导微信连续包月和 Partner VIP 直充，并参与既有会员订单与支付体系建设；将 Contract、Order、Payment 与用户权益分层建模，通过状态校验、唯一业务键和可重试补偿处理重复扣款、重复发放及跨服务部分失败；渠道侧组合 AAD App Role、Partner 签名与外部订单幂等，并分离充值和通知状态，避免 Callback 失败触发重复履约。连续包月上线后自动续费 SKU 占总订单 **86%**，ARPPU 环比提升 **58%**。
- 身份会话与账号生命周期**：主导 Refresh Token、设备会话与账号软删除重构，以 RSA JWT + UserDeviceRecord 支持设备级撤销、配额控制及 Legacy Token 渐进迁移，将设备 Token 记录从按日增长优化为按活跃设备稳定收敛；设计可恢复删除状态机、持久化事件和周期补扫，治理重复删号、任务中断及跨身份服务清理。基于高频刷新基线和百万消息压测，采用“同步保活、异步批量清理”降低在线数据库压力。

早期项目：XPay (2022.06–2022.10) | PC Manager (2022.11–2024.06)

个人工程实践与开源贡献

Graph Rail | Graph-Based Agent Harness | 个人工程实践

面向复杂软件开发任务构建 Agent 工程 Harness，以有向图组织 Build、Review、Test、Gate 等节点，并在概率型 Agent 之外建立确定性的流程控制、独立评价与证据验证机制，解决 Agent“声称完成”与任务“真实完成”之间的可信度问题。

- 独立评价机制**：贯彻“做事的人不评价自己”，拆分 Implementer、Reviewer、Test Designer 三责为隔离上下文的 Agent；Test Executor 不委托给 Agent——测试命令由 Harness 进程亲自执行并绑定证据，杜绝“Agent 声称已跑测试”。
- 测试真实性验证**：不将 Agent 生成的测试报告视为测试已执行，由 Harness 亲自运行测试命令，并对测试计划、执行命令和结果进行 Hash 绑定及 HMAC 链式签名；拒绝缺少 Provenance、自行编写、执行后修改以及“0 用例 PASS”的无效证据。
- 确定性控制面**：将流程建模为带 PASS / ITERATE / FAIL 边的有向图，通过 Handshake、合法状态转换、机械 Gate、Fail-closed 校验和循环上限约束 Agent，使流程推进、失败返工与终止不依赖模型自我声明。

开源贡献 | Hermes Agent / Multica

围绕 Agent Runtime 安全、工具调用可靠性与多租户数据边界开展开源贡献；Hermes Agent 多项实现经维护者 cherry-pick / salvage 后进入主线并保留作者信息，Multica 5 个 PR 直接合并。

- Hermes Agent | 凭据生命周期安全**：发现 Gateway 将短期配对码以明文持久化，使具备本地文件读取能力的进程可绕过交互流程直接获取凭据；设计随机 Salt + SHA-256 存储、常量时间比较及旧格式兼容迁移，并以不可恢复明文、Salt 唯一性和新旧数据互操作测试锁定安全不变量，相关实现经维护者整理后进入主线 ([#8056](#))。
- Hermes Agent | 流式 Tool Call 完整性**：定位部分 API Router 改写 finish_reason 后，截断的 JSON 参数可能绕过长度检测并进入工具执行链的问题；将判断依据从不可靠的结束元数据下沉至 Tool Call 结构本身，独立校验参数闭合与可解析性，在执行前阻断残缺调用，避免无效重试及错误工具执行，提交被 cherry-pick 至主线 ([#7693](#))。
- Multica | 多租户与内容安全边界**：推动 5 个 PR 合并，重点将 Workspace 校验从上层 Handler 下沉至 SQL DELETE / UPDATE 条件，并以跨租户反向用例锁定“错误 Workspace 不得修改数据”的不变量；同时修复 SVG 内联渲染导致的 Stored XSS，将恶意内容在共享存储出口统一降级为附件下载，实现对调用方遗漏检查的纵深防御 ([#3027](#)、[#3023](#))。